

Chuyên đề: Hệ Phương trình Tuyến tính trong Biến đổi Cấu hình & Trò chơi Lưới / Xoay lật

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

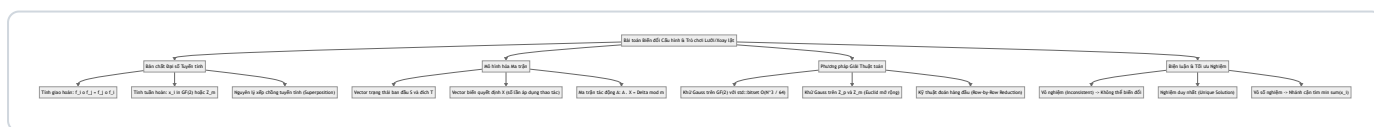
1. Tổng quan & Bản đồ Tư duy (Mindmap)

Trong lập trình thi đấu (Competitive Programming), các bài toán về **biến đổi cấu hình** (Configuration Transformations), **trò chơi lật bảng / lật đèn** (Lights Out, Flip Game), và **xoay mảnh ghép / bánh răng** (Rotation & Wheel Puzzles) thường có không gian trạng thái khổng lồ ($2^{N \times M}$ hoặc K^N), khiến các thuật toán tìm kiếm truyền thống như BFS, DFS hay Quay lui (Backtracking) bị tràn thời gian (TLE) ngay từ kích thước nhỏ ($N, M \geq 5$).

Tuy nhiên, nếu hệ thống thỏa mãn hai tính chất cốt lõi:

- Tính giao hoán (Commutativity):** Thứ tự áp dụng các phép thao tác không làm thay đổi kết quả cuối cùng.
- Tính tuần hoàn / tự triệt tiêu (Periodicity / Involution):** Áp dụng một thao tác m lần sẽ đưa hệ thống về trạng thái ban đầu (ví dụ: lật 2 lần trên $\mathbb{F}_2$, xoay 90° 4 lần trên $\mathbb{Z}_4$).

Toàn bộ bài toán tìm kiếm có thể được **chuyển hóa hoàn toàn thành Hệ Phương trình Tuyến tính** (System of Linear Equations) trên trường nhị phân $\mathbb{F}_2$ (với phép toán XOR) hoặc trên vành số nguyên $\mathbb{Z}_m$.



2. Cơ sở Đại số Tuyến tính & Bản chất Mô hình hóa

2.1. Điều kiện Cần và Đủ để Tuyến tính hóa

Giả sử hệ thống có N vị trí/ô cần quan sát và M thao tác có thể thực hiện:

- Mỗi thao tác $j \in \{1, \dots, M\}$ khi thực hiện sẽ làm biến đổi trạng thái của một tập các vị trí.
- Gọi x_j là số lần ta áp dụng thao tác thứ j .

Một bài toán biến đổi cấu hình đưa về được hệ phương trình tuyến tính khi và chỉ khi:

- Tính cộng dồn độc lập:** Tác động của thao tác lên vị trí i không phụ thuộc vào trạng thái hiện tại của hệ thống.
- Tính giao hoán:** Kết quả của việc thực hiện thao tác A rồi thao tác B đồng nhất với việc thực hiện B rồi A .

3. **Tính hữu hạn theo chu kỳ Modulo:** Tồn tại số nguyên dương $m \geq 2$ sao cho thao tác j thực hiện m lần tương đương với không thực hiện gì ($x_j \in \mathbb{Z}_m$).

- Với trò chơi lật mặt (úp/ngửa, bật/tắt): $m = 2 \implies x_j \in \{0, 1\}$ trên trường $\mathbb{F}_2$.
- Với phép xoay hình 90° : $m = 4 \implies x_j \in \{0, 1, 2, 3\}$ trên $\mathbb{Z}_4$.
- Với bánh xoay 3 trạng thái: $m = 3 \implies x_j \in \{0, 1, 2\}$ trên $\mathbb{Z}_3$.

2.2. Thiết lập Hệ Phương trình Tuyến tính

Đặt:

- S_i : Trạng thái ban đầu của vị trí i ($i = 1, \dots, N$).
- T_i : Trạng thái mong muốn (đích) của vị trí i .
- $\Delta_i = (T_i - S_i) \pmod m$: Sự biến thiên cần bù đắp tại vị trí i .
- $A_{i,j}$: Mức độ tác động của thao tác j lên vị trí i trong một lần thực hiện.

Tại mỗi vị trí i , tổng biến thiên do tất cả M thao tác gây ra phải đúng bằng Δ_i :

$$\sum_{j=1}^M A_{i,j} \cdot x_j \equiv \Delta_i \pmod m \quad (\forall i = 1, \dots, N)$$

Dưới dạng ma trận:

$$A \cdot X \equiv \Delta \pmod m$$

Trong đó:

- $A \in \mathbb{Z}_m^{N \times M}$ là **Ma trận tác động** (Incidence / Transition Matrix).
- $X = [x_1, x_2, \dots, x_M]^T \in \mathbb{Z}_m^M$ là **Vector biến quyết định**.
- $\Delta = [\Delta_1, \Delta_2, \dots, \Delta_N]^T \in \mathbb{Z}_m^N$ là **Vector sai số mục tiêu**.

⚡ 3. Khử Gauss trên Trường Nhị phân $\mathbb{F}_2$ (Bitwise Gaussian Elimination)

Khi $m = 2$, hệ thống hoạt động trên trường Galois $\mathbb{F}_2 = (\text{GF}(2), \oplus, \wedge)$.

- Phép cộng và trừ đồng nhất với phép **XOR** ($\oplus$): $1 \oplus 1 = 0, 1 \oplus 0 = 1$.
- Phép nhân đồng nhất với phép **AND** ($\wedge$): $1 \wedge 1 = 1, 1 \wedge 0 = 0$.

3.1. Tối ưu hóa Khử Gauss bằng `std::bitset`

Trong C++, việc biểu diễn mỗi phương trình bằng một hàng `bitset<MAX_VARS>` cho phép:

- Phép khử hàng ($R_i \leftarrow R_i \oplus R_k$) thực hiện trên 64 biến cùng lúc chỉ trong **1 chu kỳ xung nhịp CPU** (`^=` `bitset`).
- Giảm độ phức tạp thời gian từ $O(N^2M)$ xuống $O\left(\frac{N^2M}{64}\right)$.

```

const int MAXN = 505;
int gaussianEliminationGF2(int n, int m, std::vector<std::bitset<MAXN>>& a, std::bitset<MAXN>& ans)
{
    int rank = 0;
    std::vector<int> pivot_col(n, -1);

    for (int col = 0; col < m && rank < n; ++col) {
        int pivot = rank;
        while (pivot < n && !a[pivot].test(col)) {
            pivot++;
        }
        if (pivot == n) continue;

        std::swap(a[rank], a[pivot]);
        pivot_col[rank] = col;

        for (int i = 0; i < n; ++i) {
            if (i != rank && a[i].test(col)) {
                a[i] ^= a[rank];
            }
        }
        rank++;
    }

    // Kiểm tra vô nghiệm: hàng có dạng [0 0 ... 0 | 1]
    for (int i = rank; i < n; ++i) {
        if (a[i].test(m)) return -1; // Vô nghiệm
    }

    // Gán nghiệm cơ sở (các biến tự do gán bằng 0)
    ans.reset();
    for (int i = 0; i < rank; ++i) {
        if (a[i].test(m)) {
            ans.set(pivot_col[i]);
        }
    }

    return m - rank; // Trả về số bậc tự do (degrees of freedom)
}

```

3.2. Biện luận Nghiệm & Không gian Nghiệm (Solution Space)

Gọi $r = \text{rank}(A)$ là hạng của ma trận hệ số:

1. **Vô nghiệm** ($r < N$ và tồn tại $a_{i,m} = 1$ với $i \geq r$):

- Không tồn tại bất kỳ chuỗi thao tác nào có thể chuyển từ trạng thái S sang T .

2. **Nghiệm duy nhất** ($r = M$ và hệ tương thích):

- Tồn tại đúng một phương án thực hiện thao tác duy nhất.

3. **Vô số nghiệm** ($r < M$ và hệ tương thích):

- Hệ có $k = M - r$ biến tự do (Free Variables).

- Có chính xác 2^k phương án hợp lệ.

- **Bài toán tìm cực tiểu thao tác:** $\min \sum_{j=1}^M x_j$.

- Nếu $k \leq 20$: Ta có thể duyệt quay lui (Backtracking) hoặc duyệt 2^k trạng thái của các biến tự do để tìm nghiệm có $\sum x_j$ nhỏ nhất trong $O(2^k \cdot M)$.
- Nếu k lớn: Bài toán tìm vector nhị phân có trọng số Hamming nhỏ nhất thuộc không gian affine là bài toán NP-hard tổng quát, nhưng trong CP thường đồ thị có cấu trúc đặc biệt hoặc k rất nhỏ.

4. Khử Gauss trên Vành Modulo $\mathbb{Z}_m$ (Bài toán Xoay hình 4 Hướng)

4.1. Bài toán Xoay Mảnh ghép 90° ($m = 4$)

Xét bài toán mỗi ô là một khối có thể xoay theo 4 hướng:

- 0: Hướng Bắc (0°)
- 1: Hướng Đông (90°)
- 2: Hướng Nam (180°)
- 3: Hướng Tây (270°)

Mỗi lần kích hoạt thao tác j , các ô lân cận được xoay theo chiều kim đồng hồ 90° (cộng thêm 1 (mod 4)). Phương trình tại mỗi ô i :

$$\sum_{j=1}^M A_{i,j} \cdot x_j \equiv \Delta_i \pmod{4}$$

Cảnh báo / Bẫy:

Vì $m = 4$ không phải là số nguyên tố, vành $\mathbb{Z}_4$ tồn tại ước của 0 ($2 \cdot 2 \equiv 0 \pmod{4}$), do đó phần tử 2 **không có nghịch đảo nhân** trong $\mathbb{Z}_4$. Ta không thể chia trực tiếp như trên trường số thực hay trường nguyên tố!

Thuật toán Khử Gauss trên vành $\mathbb{Z}_m$ tổng quát:

Để khử hai hàng R_1 và R_2 tại cột c với các hệ số $a = R_1[c]$ và $b = R_2[c]$:

1. Sử dụng thuật toán Euclid mở rộng để tìm $g = \gcd(a, b, m)$ và các hệ số Bézout u, v sao cho:

$$u \cdot a + v \cdot b \equiv g \pmod{m}$$

2. Thay thế hàng R_1 bởi $uR_1 + vR_2$, và hàng R_2 bởi $\frac{a}{g}R_2 - \frac{b}{g}R_1$.
3. Quá trình này tương đương với phép biến đổi đưa ma trận về dạng chuẩn Hermite (Hermite Normal Form).

5. Kỹ thuật Đoán Hàng Đầu (Row-by-Row Reduction) cho Lưới $R \times C$

Với trò chơi **Lights Out** kinh điển trên lưới kích thước $R \times C$:

- Khi bấm vào ô (r, c) , ô đó và 4 ô kề cạnh đổi trạng thái.
- Hệ phương trình tuyến tính có $N = R \cdot C$ phương trình và $M = R \cdot C$ ẩn. Khử Gauss tổng quát tốn $O\left(\frac{(R \cdot C)^3}{64}\right)$.

Tuy nhiên, ta có tính chất **lan truyền đơn ánh (Uniqueness of Propagation)**:

- Nếu trạng thái bấm của hàng 1 đã được cố định $(x_{1,1}, x_{1,2}, \dots, x_{1,C})$, thì để đưa các bóng đèn ở hàng 1 về trạng thái tắt hoàn toàn, hành động ở hàng 2 bị **xác định duy nhất**:

$$x_{2,c} = \text{trạng thái còn lại của ô } (1, c)$$

- Tương tự, hành động ở hàng 3 bị xác định duy nhất bởi hàng 2, ..., hành động ở hàng R bị xác định duy nhất bởi hàng $R - 1$.

- Cuối cùng, chỉ cần kiểm tra xem hàng R có hoàn toàn tắt hay không!

```
Hàng 1:  [ ? ] [ ? ] [ ? ] [ ? ]    <-- Duyệt  $2^C$  khả năng (Bitmask)
           |   |   |   |
Hàng 2:  [ ! ] [ ! ] [ ! ] [ ! ]    <-- Xác định đơn ánh bởi Hàng 1
           |   |   |   |
Hàng 3:  [ ! ] [ ! ] [ ! ] [ ! ]    <-- Xác định đơn ánh bởi Hàng 2
```

- **Độ phức tạp:** $O(2^C \cdot R \cdot C)$.
- Nếu $C \leq 20$ và R lớn (hoặc $\min(R, C) \leq 20$), phương pháp này chạy nhanh gấp hàng trăm lần so với Khử Gauss đầy đủ!

6. Trace Thuật toán Từng bước: Trò chơi Lights Out 3×3

Xét lưới 3×3 với ma trận trạng thái ban đầu đều là 1 (tất cả các đèn đang bật). Cần bấm để tắt cả đèn chuyển sang 0 (tắt).

Bước 1: Đánh số 9 ô và lập vector mục tiêu

Thứ tự các ô:

$1(1, 1), 2(1, 2), 3(1, 3), 4(2, 1), 5(2, 2), 6(2, 3), 7(3, 1), 8(3, 2), 9(3, 3)$.

Mỗi ô ban đầu là 1, cần đưa về 0 $\implies \Delta_i = 1 \pmod{2}$ với mọi $i = 1, \dots, 9$.

Bước 2: Ma trận tác động $A_{9 \times 9}$ trên $\mathbb{F}_2$

Mỗi hàng i biểu thị các ô có thể tác động lên ô i (chính nó và các ô kề):

$$\hat{O} 1: x_1 \oplus x_2 \oplus x_4 = 1$$

$$\hat{O} 2: x_1 \oplus x_2 \oplus x_3 \oplus x_5 = 1$$

$$\hat{O} 3: x_2 \oplus x_3 \oplus x_6 = 1$$

$$\hat{O} 4: x_1 \oplus x_4 \oplus x_5 \oplus x_7 = 1$$

$$\hat{O} 5: x_2 \oplus x_4 \oplus x_5 \oplus x_6 \oplus x_8 = 1$$

$$\hat{O} 6: x_3 \oplus x_5 \oplus x_6 \oplus x_9 = 1$$

$$\hat{O} 7: x_4 \oplus x_7 \oplus x_8 = 1$$

$$\hat{O} 8: x_5 \oplus x_7 \oplus x_8 \oplus x_9 = 1$$

$$\hat{O} 9: x_6 \oplus x_8 \oplus x_9 = 1$$

Bước 3: Khử Gauss trên $\mathbb{F}_2$

Thực hiện các phép XOR giữa các hàng để đưa ma trận về ma trận tam giác trên / đường chéo:

- Ma trận có hạng đầy đủ: $\text{rank}(A) = 9$.

- Hệ có **nghiệm duy nhất**:

$$X = [1, 0, 1, 0, 1, 0, 1, 0, 1]^T$$

Bước 4: Kết luận nghiệm

Để tắt toàn bộ 9 đèn, ta cần bấm đúng 5 ô: Bấm tất cả 4 ô góc $(1, 1), (1, 3), (3, 1), (3, 3)$ và ô chính giữa $(2, 2)$ ($x_1 = x_3 = x_5 = x_7 = x_9 = 1$).

Tổng số thao tác: 5 lần bấm.

⚠ 7. Cạm bẫy Phòng thi & Kỹ thuật Tối ưu (Traps & Pitfalls)

- Nhầm lẫn giữa không gian nghiệm $\mathbb{F}_2$ và số thực $\mathbb{R}$:**
- Tuyệt đối không dùng kiểu `double` để khử Gauss rồi làm tròn modulo 2. Sai số dấu phẩy động sẽ làm hỏng hoàn toàn nghiệm nguyên. Luôn dùng kiểu số nguyên hoặc `std::bitset`.
- Quên kiểm tra tính tương thích khi hệ có bậc tự do:**
- Khi $\text{rank}(A) < N$, cần duyệt từ hàng `rank` đến $N - 1$: nếu có bất kỳ hàng nào toàn 0 ở phần hệ số nhưng cột tự do bên phải bằng 1 thì hệ **vô nghiệm** ngay lập tức.
- Bài toán yêu cầu nghiệm cực tiểu số bước:**
- Hệ phương trình trên $\mathbb{F}_2$ chỉ tìm ra một nghiệm đại số. Nếu đề bài yêu cầu $\min \sum x_i$, việc chỉ chọn nghiệm tự do bằng 0 có thể không phải là nghiệm cực tiểu! Phải nhánh cận trên tập biến tự do nếu số biến tự do $k \leq 20$.
- Tràn bộ nhớ khi kích thước lưới lớn:**
- Lưới $N \times M = 1000 \implies$ ma trận kích thước 1000×1000 . Lưu bằng `int a[1000][1000]` tốn 4 MB, nhưng nếu dùng `std::bitset<1005> a[1005]` chỉ tốn ≈ 125 KB, vừa tiết kiệm bộ nhớ vừa nằm gọn trong CPU L2/L3 cache giúp tăng tốc gấp hàng chục lần.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.